

Guillermo Román

guillermo.roman@upm.es

Lars-Åke Fredlund

lfredlund@fi.upm.es

Manuel Carro

mcarro@fi.upm.es

Julio García

juliomanuel.garcia@upm.es

Tonghong Li

tonghong@fi.upm.es

Francisco de Borja Lozano

francisco.lozano@upm.es

- Fechas de entrega y penalización:

Hasta el martes 29 de septiembre, 16:00 horas	0 %
Hasta el miércoles 30 de septiembre, 16:00 horas	20 %
Hasta el jueves 1 de octubre, 16:00 horas	40 %

Después la puntuación máxima será 0
- Se comprobará plagio y se actuará sobre los detectados.
- Usad las horas de tutoría para preguntar sobre programación – son oportunidades excelentes para aprender.

Entrega

- Todos los ejercicios de laboratorio se deben entregar a través de
`http://deliverit.fi.upm.es`
- El/los fichero(s) que hay que subir es/son `ActaNotasImpl.java`.
- La clase debe estar en el paquete `aed.actanotas`
- La documentación de la API de `aedlib.jar` está disponible en
`http://costa.ls.fi.upm.es/teaching/aed/docs/aedlib/`

Tarea: Implementar la interfaz ActaNotas dentro una clase ActasNotasImpl

```
public interface ActaNotas {  
    public ActaNotas addCalificacion(String nombre, String matricula,  
                                     String grupo, double nota);  
    public Calificacion getCalificacion(String matricula);  
    public ActaNotas updateCalificacion(Calificacion calificacion);  
    public ActaNotas deleteCalificacion(String matricula);  
  
    public double notaMedia();  
    public IndexedList<Pair<String,Integer>> alumnosPorGrupo();  
    public IndexedList<Calificacion>  
        getCalificaciones(Function<Calificacion,Boolean> filter,  
                          Comparator<Calificacion> cmp);  
    ...  
}
```

- La interfaz detalla métodos para almacenar las notas de alumnos de una asignatura, y realizar varias preguntas sobre la coleccion de notas.
- Documentación en ActaNotas.java y Calificacion.java.

Actas, calificaciones y notas

- Un *acta* consiste en una lista de las notas de los alumnos de una asignatura, de un año y de una convocatoria (ordinaria o extraordinaria – julio). Además un acta define una nota mínima para que un alumno sea aprobado.

```
new ActaImpl("AED", 5.0, 2026, false)
```

- Las notas en la acta son instancias de la clase Calificacion.

- ▶ La clase tiene un constructor

```
Calificacion(String nombreAlumno, String matricula,  
             String grupo, double nota)
```

- ▶ Ejemplo: `new Calificacion("Paco", "25M099", "3S1T-MI", 3.0)`
- ▶ Los métodos de la clase están documentados en el fichero `Calificacion.java`

La clase ActaNotasImpl

- La clase ActaNotasImpl debe implementar la interfaz ActaNotas:

```
public class ActaNotasImpl implements ActaNotas { ... }
```

- Podéis usar los atributos y métodos privados que consideréis necesarios. Sin embargo, **Es obligatorio guardar las calificaciones en un atributo** calificaciones **dentro la clase**:

```
private IndexedList<Calificacion> calificaciones
```

- La clase debe implementar un constructor

```
ActaNotasImpl(String asignatura, double notaMinimaAprobado,  
               int anyo, boolean esConvocatoriaExtraordinaria)
```

El constructor debe inicializar el atributo calificaciones que almacena las calificaciones y los atributos que consideréis oportuno.

La clase ActaNotasImpl

La clase debe además redefinir estos dos métodos estándar ya definidos en la clase `Object`:

- `public boolean equals(Object obj)`

El método debe devolver `true` si el parámetro `obj` es una instancia de la clase `ActaNotasImpl` con la misma asignatura, el mismo año, y la misma convocatoria.

- `public String toString()`

El método debe devolver un `String` que representa el acta (sin las clasificaciones). Usad el nombre de la asignatura, el año, e información sobre la convocatoria en el `String` devuelto por el método. El formato exacto no es importante.

El método

```
IndexedList<Pair<String,Integer>> alumnosPorGrupo()
```

devuelve una lista con pares constituidos por:

- un elemento izquierdo: el grupo – un alumno siempre pertenece a un grupo, e.g. "3S1T-MI",
- y un elemento derecho: el número de alumnos de este grupo que tiene una calificación

Usad la clase `es.upm.aedlib.Pair<X,Y>` para crear un par. La clase tiene un constructor `Pair(X x, Y y)` y los métodos

```
public X left(); // devuelve el elemento izquierdo
public Y right(); // devuelve el elemento derecho
```

El método

```
public IndexedList<Calificacion>  
    getCalificaciones(Function<Calificacion, Boolean> filter,  
                      Comparator<Calificacion> cmp);
```

devuelve una lista nueva de las calificaciones en el acta, *filtrado* por *filter* y *ordenados* por *cmp*.

- El filtro determina qué calificaciones, de entre todas las del acta, se deben devolver
- Dada una calificación *c*, si *filtro.apply(c)* devuelve *true* se incluye la calificación en la lista
- Una *Function* es una instancia de `java.util.function.Function<X,R>`. Hay que importar dicha interfaz en `ActaNotasImpl.java`
- Si el parámetro *filter* es *null* se asume que todas las calificaciones deberían estar en la lista devuelta

El método

```
public IndexedList<Calificacion>  
    getCalificaciones(Function<Calificacion, Boolean> filter,  
                      Comparator<Calificacion> cmp);
```

devuelve una lista nueva de las calificaciones en la acta, *filtrado* por *filter* y *ordenados* por *cmp*.

- La lista (de elementos filtrados) devuelto por el método debe estar ordenada por el comparador *cmp* en orden ascendiente (un elemento menor debe ir antes que un elemento más grande)
- Si el parámetro *cmp* es *null* se asume que se debe ordenar según la matrícula de una calificación
- Para crear la nueva lista podéis recorrer los elementos almacenados en el acta e, aplicando el filtro, e ir insertándolos los elementos elegidos en el orden adecuado en la lista resultado (usando para ello *cmp*)
- No es (muy) importante la eficiencia de este método

Ejemplo

```
ActaNotas a = new ActaNotasImpl("AED", 5.0, 2026, false);

a.addCalificacion("Pepe", "1", "G1", 8.0); // [ Calificacion{"Pepe",...}]
a.addCalificacion("Luis", "3", "G2", 6.5); // [ Calificacion{"Pepe",...},
                                           //   Calificacion{"Luis",...} ]

a.addCalificacion("Paco", "3", "G1", 7.0); // IllegalStateException

a.getCalificacion("1");    // returns Calificacion{"Pepe",...}

a.getCalificacion("2");    // returns null

a.addCalificacion("Paco", "5", "G1", 3.5); // [ Calificacion{"Pepe",...},
                                           //   Calificacion{"Luis",...},
                                           //   Calificacion{"Paco",...} ]
```

Ejemplo (cont.)

```
a.notaMedia();           // returns 6.0

a.alumnosPorGrupo();     // returns [ Pair("G1",2), Pair("G2",1) ]

a.getCalificaciones(null,new CompNombre());
                        // returns [ Calificacion{"Luis","3",6.5,"G2"},
                        //           Calificacion{"Paco","5",3.5,"G1"},
                        //           Calificacion{"Pepe","1",8.0,"G1"} ]

a.getCalificaciones(null,new CompNota());
                        // returns [ Calificacion{"Paco","5",3.5,"G1"},
                        //           Calificacion{"Luis","3",6.5,"G2"},
                        //           Calificacion{"Pepe","1",8.0,"G1"} ]

a.getCalificaciones((Calificacion c) -> c.nota() >= 5.0, null);
                        // returns [ Calificacion{"Pepe","1",8.0,"G1"},
                        //           Calificacion{"Luis","3",6.5,"G2"} ]

a.getCalificaciones((Calificacion c) -> c.nota() >= 5.0, new CompNombre());
                        // returns [ Calificacion{"Luis","3",6.5,"G2"},
                        //           Calificacion{"Pepe","1",8.0,"G1"} ]

a.deleteCalificacion("1"); // [ Calificacion{"Luis","3",6,"G2"},
                        //     Calificacion{"Paco","5",3,"G1"} ]

a.deleteCalificacion("1"); // IllegalStateException
```

Consejos:

- Utilizad un atributo de tipo `IndexedList<Calificacion>` para almacenar las calificaciones.
- Mantened la estructura *ordenada* usando la matrícula (recordad que `String` implementa la interfaz `Comparable`).
- Para los métodos `updateCalificacion`, `deleteCalificacion`, `getCalificacion`:

- ▶ Usad un método privado que devuelva la posición que ocupa la matrícula, o `-1` si no lo encuentra, con cabecera:

```
private int buscarMatricula (String matricula)
```

- ▶ Si la estructura está ordenada por matrícula, `buscarMatricula` puede hacer búsqueda binaria $\Rightarrow$ complejidad $O(\log(n))$ en lugar de $O(n)$.

Seguir estos consejos os permitirá conseguir mejores resultados!

Notas Generales

- El proyecto debe compilar sin errores y debe cumplirse la especificación de los métodos a completar
- Debe pasar todos los test `TesterLab1` correctamente sin mensajes de error
- **Nota:** una ejecución sin mensajes de error y que pase todas las pruebas **no** significa que la implementación sea correcta (es decir, que funcione bien para cada posible entrada)
- Todos los ejercicios se corrigen manualmente antes de dar la nota final

¡Seguir estos consejos os permitirá conseguir mejores resultados!

- Corrección
- Ausencia de código repetido con la misma funcionalidad (podéis usar métodos auxiliares para evitarlo)
- Concisión y claridad del código
- Legibilidad, incluida selección de nombres descriptivos para variables y métodos
- El código debe estar correctamente indentado y con comentarios *útiles* cuando lo veáis necesario
- Eficiencia:
 - ▶ Se valorará la complejidad computacional del código
 - ▶ Se valorará no iterar innecesariamente en los recorridos de las estructuras de datos